

EE 432

RF Engineering for Telecommunications

Lab # 9-10

FSK Receiver (Demodulator)

Name: Mohammad Chahardori (Student ID: 11632753)

Partner: Richard Osorio

Due Date: 11/14/2020

Objectives

In this lab we are supposed to understand the structure and the operation of FSK demodulator by implementing the systematic blocks and required sub-blocks in an FSK receiver using MATLAB coding. We will verify the functionality and the operation of the FSK demodulator by applying a noisy AFSK modulated signal and extracting the transmitted data using our implemented FSK receiver.

Procedure and Analysis of Data

- To check the operation of our implemented code for FSK demodulator in lab 9 we used a sample FSK modulated signal. The sample data includes 8 synchronous bits and 16-bit data coming after the synchronous bits. In lab 10 we use the provided wave file as the modulated signal which has same sync pattern, but the data is not disclosed before the demodulation. Also, the provided wave file is included amplitude noise which makes the selection of decision threshold level more important to provide the proper data at the modulator output. The provided data for lab10 has sampling rate of 4000 samples per second and its time is 0.1 second, hence $N_s = 400$. The carrier frequency is 500 Hz, and the frequency deviation is 10 Hz which results in $f_0 = 490$ Hz and $f_1 = 510$ Hz. We used:

```
[r,fs] = audioread('newwav.wav');
```

command to read this wave file into a MATLAB code.

Block Diagram of the FSK Receiver and the Output at Different Stages

The structure of the FSK receiver has been shown in figure 1.

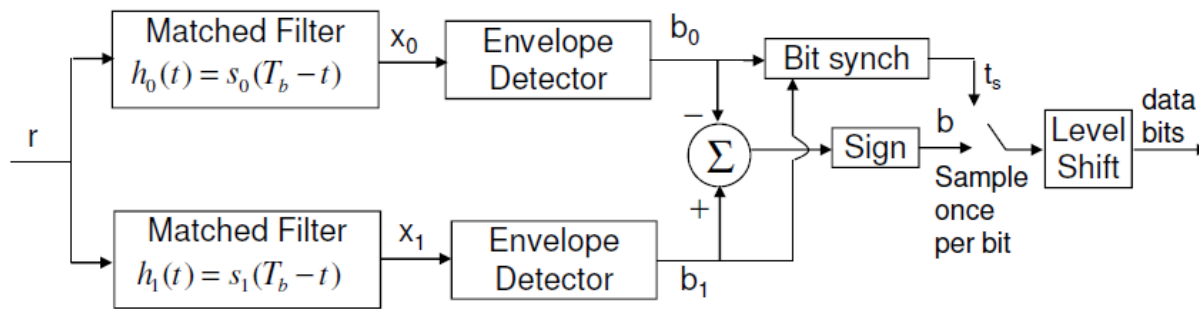


Figure 1: Block diagram of the FSK receiver and names of output signal at each intermediate stage

The input signal r is the received signal which is read from the wave file as described before. In figure 2 the received signal versus time for several points is shown. As we can see the signal frequency has variation over the time. Although it is composed mainly from two different frequencies however due to small difference between these two frequencies which are 490Hz and 510Hz and also due to the noise added to the signal it's not easy to distinguish the two different frequencies sent as 0 or 1 in this figure.

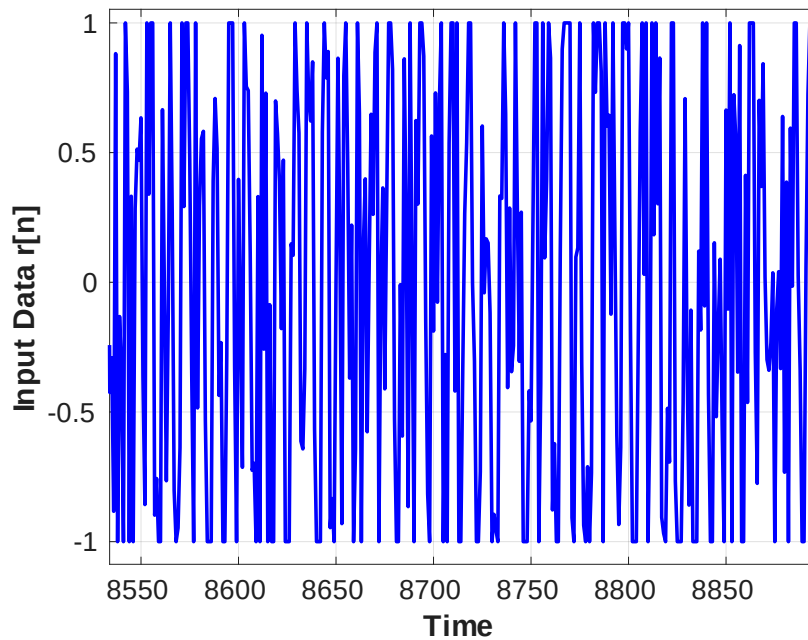


Figure 2: Received signal $r[n]$. The signal is zoomed in.

The signal spectrum regarding the received signal has shown in figure 3. in this figure we can see the main two frequencies (490Hz and 510Hz) which are used in FSK modulation and also, we can see the noise level added to the signal. the difference between the amplitude of the received signal in f_0 and f_1 is mostly because of the difference between the number of transmitted 1 and transmitted 0 in the received signal. since we have more zero compared to one the amplitude of the signal at f_0 is a little higher than the amplitude of signal in frequency f_1 .

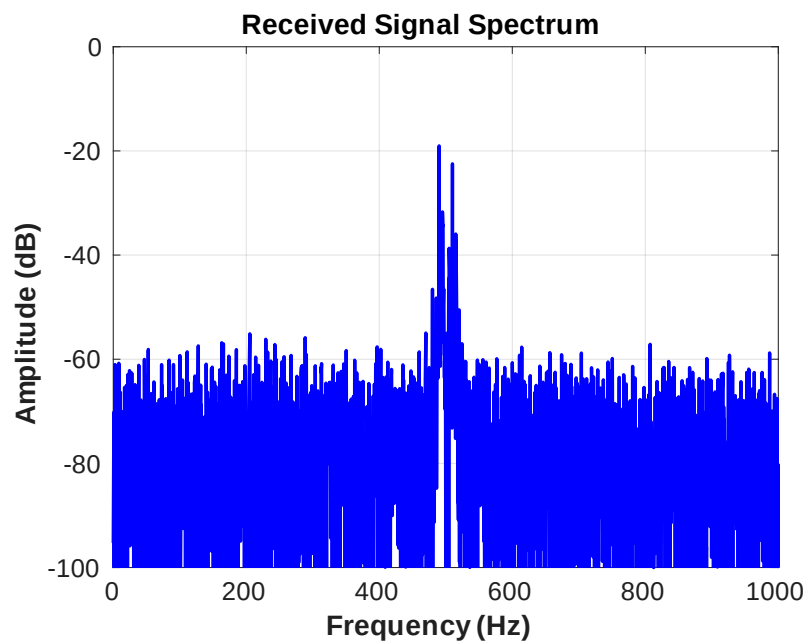


Figure 3: Spectrum of the Received signal

The operation of each block and the way we have implemented them in MATLAB is described here.

1. Matched Filter:

The matched filter is used to determine the correlation of the received signal with predefined frequencies f_0 and f_1 which is done by the convolution operation as follow:

$$s_0(T_b - t) = A \sin(2\pi f_0(T_b - t)) = A \sin\left(2\pi \frac{n}{T_b} T_b - 2\pi f_0 t\right) = -A \sin(2\pi f_0 t)$$

By similar argument,

$$s_1(T_b - t) = -A \sin(2\pi f_1 t)$$

The matched filter outputs, are given by:

$$x_i(t) = \text{conv}(r, s_i(t)) \times \frac{2}{N_s}; \quad \text{where } i=0,1$$

Output of the matched filter $x_0(t)$ and $x_1(t)$ are shown below in Figure 4.

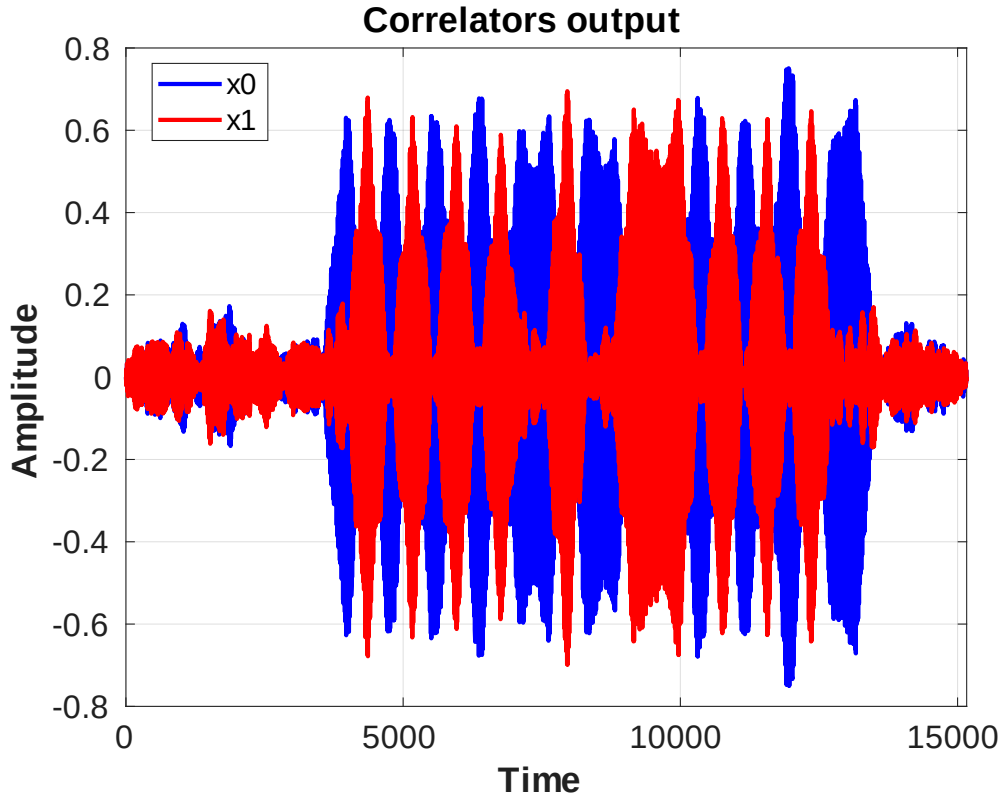


Figure 4: Received signals, x_0 and x_1 after the correlators

As we can see in figure 4 the received signal includes only noise till time (sample) around 4000, then we are receiving modulated signal and as a result the correlators output have meaningful variations. also, we can see after sample around 13000 we received only the noise again.

2. Envelope Detector:

To extract the correlated output envelope from the correlated signals we have the envelope detector after correlators in FSK receiver which consists of a rectifier and a low-pass filter as shown in figure 5.

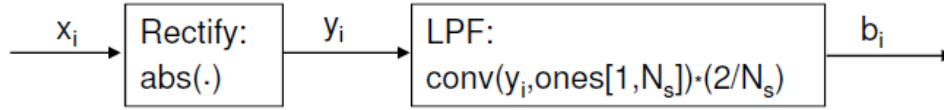


Figure 5: Envelope Detector Sub-blocks consist of Rectifier and LPF

In Matlab code to realize the rectifier we are using abs function which generate the absolute value of the signal regardless of the signal sign and also to realize the low-pass filter we are using a time domain convolution between the signal and 1's vector which works like a low pass filter in the frequency domain. In figures 6 the outputs of the envelope detector b0 and b1 are shown.

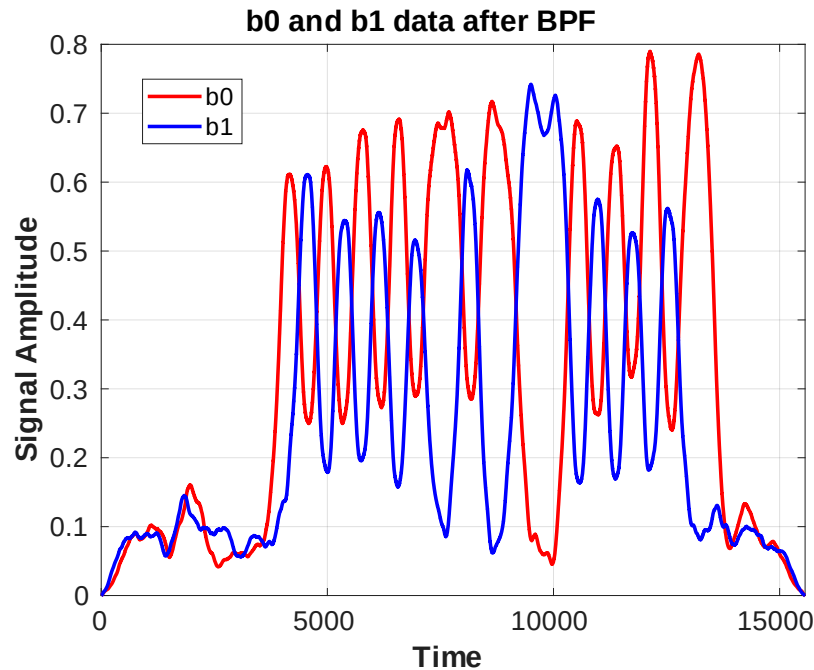


Figure 6: Received signals after the envelope detector, b0 and b1

3. Bit synchronization:

After detecting the b0 and b1 we need to use these two signals to realize when the receiver is receiving the modulated signal instead of the noise. To understand the starting time that the modulated data is receiving we have used the (b0+b1) signal. When the level of (b0+b1) signal is higher than specific value which we considered it around $\text{Max}(b0+b1)/2 = 0.5$, we can make sure that the received energy is high enough to indicate we are receiving a modulated signal (not noise). In fact, as we can see the noise level after correlator and envelope detector does not raise over 0.18 so considering 0.5 as the threshold level

provides a very good noise margin in our receiver for the current noise level of our received signal. In figure 7, the signal b_0+b_1 and the threshold level is seen.

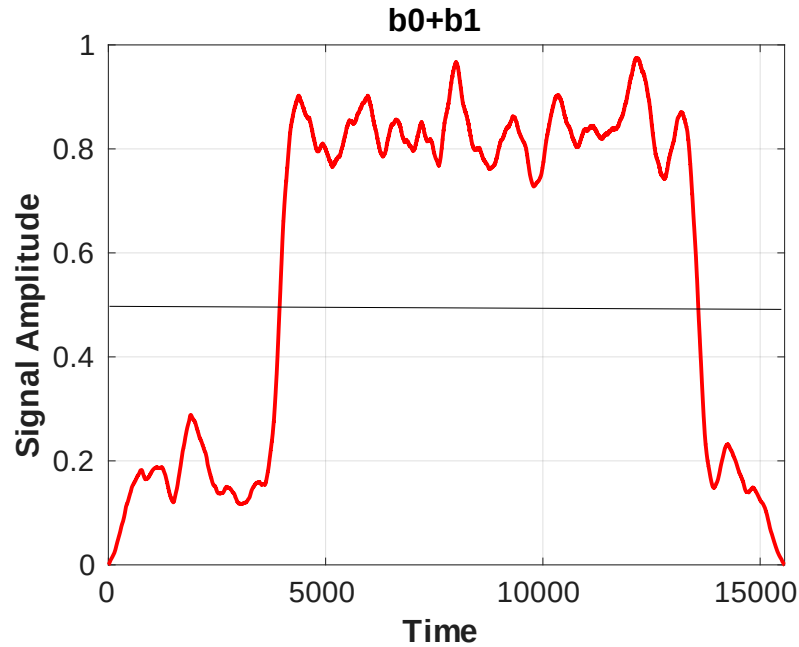


Figure 7: b_0+b_1 signal and threshold level

In figure 8 the signal b_1-b_0 which is used for sampling after synchronization is shown.

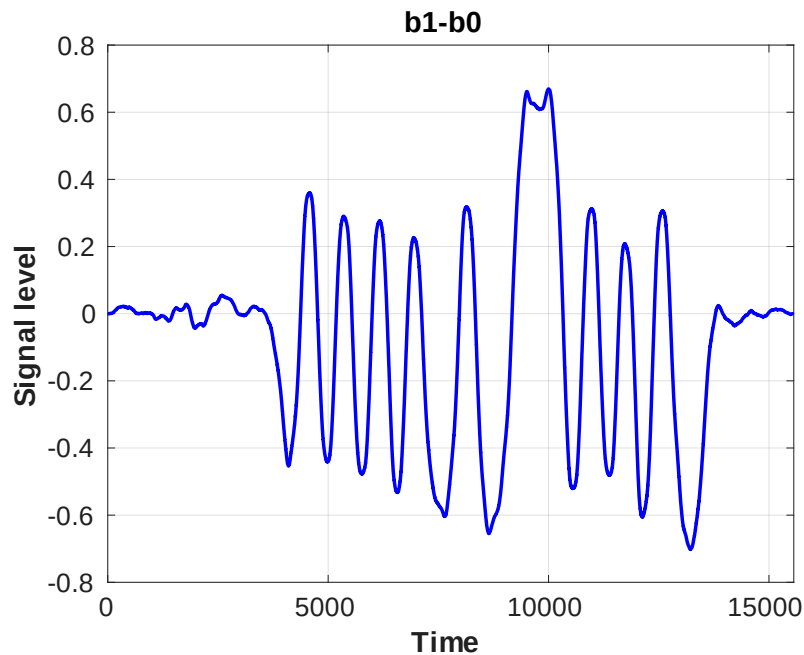


Figure 8: b_1-b_0 signal versus time (samples)

On the other hand, we need to sample the received data at the right time to recover data. Best time for sampling is when the correlation between received signal and predefined frequency is maximum. On top

of that we need to find the synchronization bit stream to find the beginning of the data stream for future synchronization. These two operations are done in synchronization section after deriving $b_0 + b_1$ as shown in figure 9.

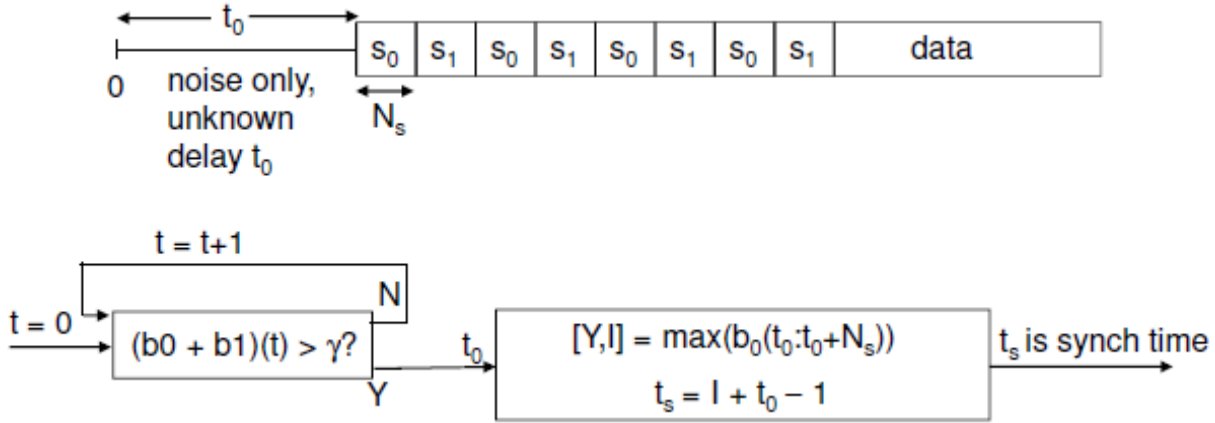


Figure 9: Bit synchronization process and sampling moment (t_s) determination

As I described before from figure 7, we set $\gamma = 0.4875$. As a result, the value of the initial delay time $t_0 = 3936$. Finally, to find the best time for sampling the data in each period of the received data after the delay t_0 , we find the time index correspond to maximum amplitude of the received signal b_0 from t_0 to $t_0 + N_s$. Using the value for delay t_0 and index l , we can calculate the best sampling time in each period. The best sampling time based on our received signal is $t_s = 4168$. Now using this sampling time (t_s) we sample the signal $b = (b_1 - b_0)$ at

$$T_{sample} = t_s + n \times N_s \text{ where } n = 1, 2, \dots$$

In figure 10 the sampled data b after the sign and level shifter operators which convert the sampled data b to corresponding binary sequence of 0's and 1's is shown. Please note this bit stream includes both the sync and data bits at the same time.

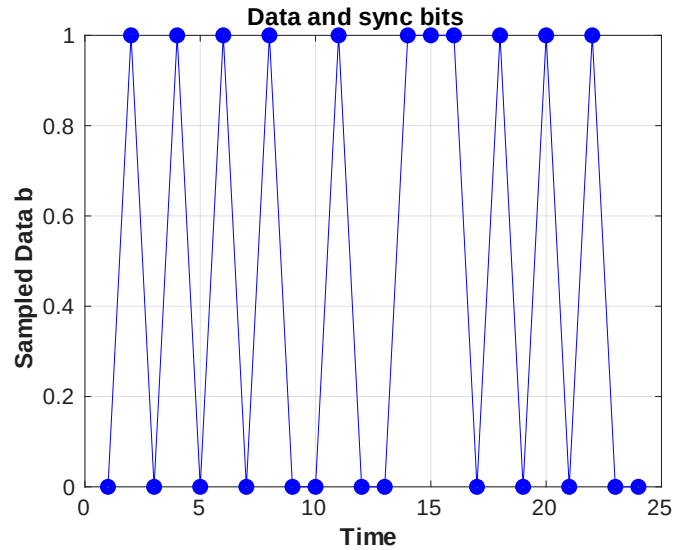


Figure 10: binary sampled data $b = (b_1-b_0)$ after sign and level shifter blocks

As seen in figure 10 the received binary signal including sync and main data is:

0 1 0 1 0 1 0 1 0 0 1 0 0 1 1 1 0 1 0 1 0 1 0 0

The last step here is to determine and separate the sync bits and data bits. To find the sync bits we compared the sampled signal b with sync sequence $[0\ 1\ 0\ 1\ 0\ 1\ 0\ 1]$. When we find the sync sequence among the received data we consider the next 16 bits as the valid data and store it in output data. The binary output data is shown in figure 11.

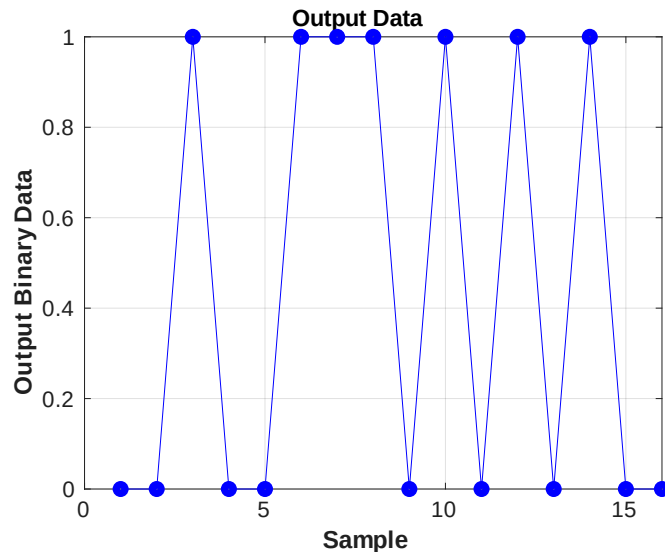


Figure 11: Output binary data

As seen in figure 11 the received binary signal is:

0 0 1 0 0 1 1 1 0 1 0 1 0 1 0

Appendix: Matlab Code for lab 9 and lab 10

Code for Lab 9:

```
%ee432 FSK lab 9
clear all
close all
clc
Tb = 0.1;
Ns = 500;
fc = 500;
df = 10;
x = linspace(0,Tb,Ns);
f0 = fc-df;
f1 = fc+df;
s0 = sin(2*pi*f0*x);
s1 = sin(2*pi*f1*x);
synch = [ s0 s1 s0 s1 s0 s1 s0 s1 ];
data = [ s0 s0 s1 s0 s1 s1 s0 s1 s0 s1 s1 s1 s1 s0 s1 s0 ];
%z = [ synch data ];
Noise_Recieved = 0.1 * rand(1,2000); % I added this rand to model the initial noise
z = [ Noise_Recieved synch data ];
%sound(z,5000);
Ts = Tb / Ns; % sampling interval
r = z;
% Matched Filter Section
x0 = conv(r,s0)*(2/Ns);
x1 = conv(r,s1)*(2/Ns);
% Envelope detection Section
y0 = abs(x0);
y1 = abs(x1);
One_vector = ones(1,Ns);
b0 = conv(y0,One_vector)*(2/Ns);
b1 = conv(y1,One_vector)*(2/Ns);
% Synchronization Section
% Finding t0
Instant_Engry_Received = b0 + b1 ;
tt = 0;
Gama = max(b0)/2;
for kk = 1 : length(Instant_Engry_Received)
    if Instant_Engry_Received(kk) < Gama
        tt = tt + 1;
    else
        t0 = tt;
    end
end
% Finding ts
[Y,I] = max(b0(t0:t0+Ns));
ts = I + t0 - 1
% generating the b vector and sign and level shifting blocks
b_vector_unsampled = b1 - b0;
for jj = 1 : length (b_vector_unsampled)
    if b_vector_unsampled( jj) > 0;
```

```

        b_vector_unsampled_sign (jj)= 1;
    elseif b_vector_unsampled( jj) < 0;
        b_vector_unsampled_sign (jj)= 0;
    end
end
b_sampled = b_vector_unsampled_sign(ts : Ns :ts + 24 * Ns) % Sampling Section
%b_sampled = b_vector_unsampled_sign(ts : Ns :end) % Sampling Section
% Finding Sync pattern
for kk = 1 : length (b_sampled) - 8
    if ( b_sampled( kk : kk+7) == [0 1 0 1 0 1 0 1]);
        T_data = kk + 8;
    else
    end
end
data_output = b_sampled (T_data:T_data + 15)
figure(1)
plot(b1)
figure(2)
stem(data_output)

```

Matlab Code for lab 10:

```
%ee432 FSK lab10
clear all
close all
clc
Tb = 0.1;
Ns = 400;
fc = 500;
df = 10;
x = linspace(0,Tb,Ns);
f0 = fc-df;
f1 = fc+df;
s0 = sin(2*pi*f0*x);
s1 = sin(2*pi*f1*x);
[r,fs] = audioread('newwav.wav');
%sound(r,5000);
Ts = Tb / Ns; % sampling interval
% Matched Filter Section
x0 = conv(r,s0)*(2/Ns);
x1 = conv(r,s1)*(2/Ns);
%+++++
%+++++
% Envelope detection Section
y0 = abs(x0);
y1 = abs(x1);
One_vector = ones(1,Ns);
b0 = conv(y0,One_vector)*(2/Ns);
b1 = conv(y1,One_vector)*(2/Ns);
%+++++
%+++++
% Synchronozation Section and finding t0 and ts
% Finding t0
Instant_Enrgy_Received = b0 + b1;
tt = 0;
Gama = max(b0+b1)/2
for kk = 1 : length(Instant_Enrgy_Received)
    if Instant_Enrgy_Received(kk) < Gama
        tt = tt + 1;
    else
        t0 = tt;
    end
end
t0
% Finding ts
[Y,I] = max(b0(t0:t0+Ns));
ts = I + t0 - 1
%+++++
%+++++
% generating b vector and sign and level shifting blocks
b_vector_unsampled = b1 - b0;
```

```

for jj = 1 : length (b_vector_unsampled)
    if b_vector_unsampled( jj) > 0;
        b_vector_unsampled_sign (jj)= 1;
    elseif b_vector_unsampled( jj) < 0;
        b_vector_unsampled_sign (jj)= 0;
    end
end
%+++++
%+++++
% Sampling Section
b_sampled = b_vector_unsampled_sign(ts : Ns :end); % Sampling b signal at ts+n*Ns
b_sampled_binary = b_sampled(1:8+16)
%+++++
%+++++
% Finding Sync pattern and output data
for kk = 1 : length (b_sampled) - 8
    if ( b_sampled( kk : kk+7) == [0 1 0 1 0 1 0 1]);
        T_data = kk + 8;
    else
    end
end
data_output = b_sampled (T_data:T_data + 15)

%+++++
%          plotting the outputs of each blocks
%+++++
figure(2)
plot( r,'-b','LineWidth',2)
xlabel('Time','fontweight','bold','fontsize',16);
ylabel('Input Data r[n]','fontweight','bold','fontsize',16)
set(gca,'FontSize',15)
grid on
%+++++
%+++++
figure(3)
[Pxx,F] = periodogram(r,hanning(length(r)),2^20,Ns/Tb);
plot( F,db(Pxx),'-b','LineWidth',2)
xlabel('Frequency (Hz)','fontweight','bold','fontsize',16);
ylabel('Amplitude (dB)','fontweight','bold','fontsize',16)
set(gca,'FontSize',15)
grid on
title('Received Signal Spectrum')
xlim([0 1000])
ylim([-100 0])
%+++++
%+++++
figure(4)
plot( x0,'-b','LineWidth',2)
xlabel('Time','fontweight','bold','fontsize',16);
ylabel('Amplitude','fontweight','bold','fontsize',16)
set(gca,'FontSize',15)
grid on

```

```

hold on;
plot(x1, '-r', 'LineWidth', 2)
title('Correlators output')
legend('x0', 'x1', 'location', "best" )
%+++++
%+++++
figure(6)
plot(b0, '-r', 'LineWidth', 2)
xlabel('Time', 'fontweight', 'bold', 'fontsize', 16);
ylabel('Signal Amplitude', 'fontweight', 'bold', 'fontsize', 16)
set(gca, 'FontSize', 15)
grid on
hold on;
plot(b1, '-b', 'LineWidth', 2)
title('b0 and b1 data after BPF')
legend('b0', 'b1', 'location', "best" )
%+++++
%+++++
figure(7)
plot(Instant_Engry_Received, '-r', 'LineWidth', 2)
xlabel('Time', 'fontweight', 'bold', 'fontsize', 16);
ylabel('Signal Amplitude', 'fontweight', 'bold', 'fontsize', 16)
set(gca, 'FontSize', 15)
grid on
hold on;
%plot(ones(length(Instant_Engry_Received))* Gama, '-b', 'LineWidth', 2)
title('b0+b1')
%+++++
%+++++
figure(8)
plot( b_vector_unsampled, '-b', 'LineWidth', 2)
xlabel('Time', 'fontweight', 'bold', 'fontsize', 16);
ylabel('Signal level', 'fontweight', 'bold', 'fontsize', 16)
set(gca, 'FontSize', 15)
grid on
hold on;
title('b1-b0')
%+++++
%+++++
figure(10)
plot(b_sampled(1:8+16), '-b', 'LineWidth', 0.1)
xlabel('Time', 'fontweight', 'bold', 'fontsize', 16);
ylabel('Sampled Data b', 'fontweight', 'bold', 'fontsize', 16)
set(gca, 'FontSize', 15)
grid on
hold on;
plot(b_sampled(1:8+16), 'ob', 'LineWidth', 5)
title('Data and sync bits')
%+++++
%+++++
figure(11)
plot(data_output, '-b', 'LineWidth', 0.1)

```

```
xlabel('Sample','fontweight','bold','fontsize',16);  
ylabel('Output Binary Data','fontweight','bold','fontsize',16)  
set(gca,'FontSize',15)  
grid on  
hold on;  
plot(data_output,'ob','LineWidth',5)  
title('Output Data','fontsize',16)
```